Computer Programming 1

Data Types: Stream

(15-09-2026)

Till now, we have used "*cout* <<"…

```
cout << " (1) Hello World " << endl;

cout << " (2) Hello World"; cout << " Hello World" << endl;

cout << " (3) Hello World"<< " Hello World" << endl;

cout << " (4) Hello World \n"; cout << " Hello World" << endl;

cout << " (5) Hello \n World" << endl;

cout << " (6) Hello \n World"; cout << " Hello World" << endl;

int x=10; cout << "(7) "<< x << " " << (x*2)+3 << endl;
```

```
(1) Hello World
(2) Hello World Hello World
(3) Hello World Hello World
(4) Hello World
 Hello World
(5) Hello
 World
(6) Hello
 World Hello World
(7) 10 23
```

The functions operating on the streams are in an input/output library to be imported with the directive:

```
#include <iostream>
```

Till now, we have used "*cout* <<"…

```
cout << " (1) Hello World " << "\n";

cout << " (2) Hello World"; cout << " Hello World" << "\n";

cout << " (3) Hello World"<< " Hello World" << "\n";

cout << " (4) Hello World \n"; cout << " Hello World" << "\n";

cout << " (5) Hello \n World" << "\n";

cout << " (6) Hello \n World"; cout << " Hello World" << "\n";

int x=10; cout << "(7) "<< x << "  " << (x*2)+3 << "\n";
```

```
(1) Hello World
(2) Hello World Hello World
(3) Hello World Hello World
(4) Hello World
 Hello World
(5) Hello
 World
(6) Hello
 World Hello World
(7) 10 23
```

The functions operating on the streams are in an input/output library to be imported with the directive:

```
#include <iostream>
```

Outline

- Streams

What is a *stream*?

- A program communicates with the outside world through streams
- A stream is a logical structure consisting of a sequence of characters (theoretically infinite sequence), that ends with a special character
- The streams are associated (with appropriate commands) to the physical devices connected to the computer (keyboard, video) or files resident on the memory

- Pre-defined streams in C++:
  cin (stream standard input)
  cout (stream standard output)
  cerr (stream standard error)
- The functions operating on the streams are in an input/output library to be imported with the directive:
  #include <iostream>

Output *stream*

- The output stream (cout) is the one used by the program to write data
  - It is typically associated to the monitor
- To write data the following expression must be used

  stream << expression;
- The writing instruction implies:
  1) computation of the value of the expression
  2) conversion in a sequence of characters
  3) transfer/move of the sequence of characters to the stream

Note

- The predefined constant endl corresponds to '\n', for which a new line is initialized with the cursor in the first column

Multiple writing

- The writing instruction has a general format that allows for multiple writing, in the following format

  cout << expression1 << expression2 << …

- E.g.,

  cout << x << y << endl;

  corresponds to:

  ```
  cout << x;
  cout << y;
  cout << endl;
  ```

Example of use of output operations

{../L3-4_EXAMPLE_stream/example_count0.cc}

{../L3-4_EXAMPLE_stream/example_count.cc}

Example with multiple output

{../L3-4_EXAMPLE_stream/example_count_multiple.cc}

Output stream – examples (2)

```
using namespace std;
#include <iostream>

int main()
{
 cout << "Hello World" << endl;  // output instruction
 return 0;
}
```

```
using namespace std;
#include <iostream>

int main()
{
 cout << "Hel" << "lo Wor" << "ld" << endl; // output instruction
 return 0;
}
```

```
using namespace std;
#include <iostream>

int main()
{
 cout << "Hello World " << " 2024 "  <<  endl;
 return 0;
}
```

```
using namespace std;
#include <iostream>

int main()
{
 int x=2024;
 cout << "Hello World " <<  x <<  endl;
 return 0;
}
```

```
$ g++ example_count0.cc
$ ./a.out
Hello World 2024
```

{../L3-4_EXAMPLE_stream/**example_count0.cc**}

Input *stream*

- The standard input stream (cin) is the one from which the program fetches data
  - It is typically associated to the keyboard
- To collect data, the read instruction is used:

  stream >> expression;

  Where expression must be an expression with an address (a variable)
- The reading expression implies:
  1) get the sequence of characters from the stream
  2) convert of the sequence of characters into a value
  3) assign the value to the variable

← Possible critical step

Reading a character from *cin*

Let's consider the instruction cin >> x, where x is of type char

□□□□a□□-14.53□□728□□ …



(where, ⬆ is the cursor and □ is a space)

- If the current position pointed by the cursor represents a space:
  1. the cursor moves forward in order to find a character that is not a space
- If the character pointed by the cursor does not represent a space:
  1. the character is picked up
  2. the character is assigned to the variable x
  3. the pointer moves to the next one

Reading a number from *cin*

Let's consider the instruction cin >> x, where x is of type int

- If the current position pointed by the cursor is a space:
  - it is as the previous case
- If the sequence of characters is interpretable as a number compatible with the type of x:

  □□□□a□□-14.53□□728□□ …

  

  1. the sequence is picked up
  2. the sequence is converted in the corresponding value (e.g., the integer 728)
  3. the converted value is assigned to the variable x
  4. the pointer moves to the next one
- Otherwise, the picking up operation does not hold and the stream goings to the error state
  - **Attention: use specific instructions (next slide) to manage the error state**

Multiple reading

- The instruction allows multiple readings in the form:

  cin >> var1 >> var2 …

- E.g., if x, y, and z are respectively: char, double and int:

  cin >> x >> y >> z;

  It correponds to:

  cin >> x;

  cin >> y;

  cin >> z;

□□□□a□□-14.53□□728□□ …



Input stream – examples (1)

```
using namespace std;
#include <iostream>

int main()
{
  char x;
  float y;
  int z;

  cout << "insert a character, a real, and an integer" << endl;
  cin >> x;
  cin >> y;
  cin >> z;
  cout << "x = " << x << ", y = " << y << ", z = " << z << endl;
  return 0;
}
```



{../L3-4_EXAMPLE_stream/example_cin.cc}

Input stream – examples (1)

```
using namespace std;
#include <iostream>

int main()
{
  char x;
  float y;
  int z;

  cout << "insert a character, a real, and an integer" << endl;
  cin >> x;
  cin >> y;
  cin >> z;
  cout << "x = " << x << ", y = " << y << ", z = " << z << endl;
  return 0;
}
```

```
$ ./a.out
```

{../L3-4_EXAMPLE_stream/example_cin.cc}

Input stream – examples (1)

```
using namespace std;
#include <iostream>

int main()
{
  char x;
  float y;
  int z;

  cout << "insert a character, a real, and an integer" << endl;
  cin >> x;
  cin >> y;
  cin >> z;
  cout << "x = " << x << ", y = " << y << ", z = " << z << endl;
  return 0;
}
```

```
$ ./a.out
```

{../L3-4_EXAMPLE_stream/example_cin.cc}

Input stream – examples (1)

```
using namespace std;
#include <iostream>

int main()
{
  char x;
  float y;
  int z;

  cout << "insert a character, a real, and an integer" << endl;
  cin >> x;
  cin >> y;
  cin >> z;
  cout << "x = " << x << ", y = " << y << ", z = " << z << endl;
  return 0;
}
```

```
$ ./a.out
insert a character, a real, and an integer
```

{../L3-4_EXAMPLE_stream/example_cin.cc}

Input stream – examples (1)

```
using namespace std;
#include <iostream>

int main()
{
  char x;
  float y;
  int z;

  cout << "insert a character, a real, and an integer" << endl;
  cin >> x;
  cin >> y;
  cin >> z;
  cout << "x = " << x << ", y = " << y << ", z = " << z << endl;
  return 0;
}
```



```
$ ./a.out
insert a character, a real, and an integer
c
```

{../L3-4_EXAMPLE_stream/example_cin.cc}

Input stream – examples (1)

```
using namespace std;
#include <iostream>

int main()
{
  char x;
  float y;
  int z;

  cout << "insert a character, a real, and an integer" << endl;
  cin >> x;
  cin >> y;
  cin >> z;
  cout << "x = " << x << ", y = " << y << ", z = " << z << endl;
  return 0;
}
```

```
$ ./a.out
insert a character, a real, and an integer
c 1.1
```

{../L3-4_EXAMPLE_stream/example_cin.cc}

Input stream – examples (1)

```
using namespace std;
#include <iostream>

int main()
{
  char x;
  float y;
  int z;

  cout << "insert a character, a real, and an integer" << endl;
  cin >> x;
  cin >> y;
  cin >> z;
  cout << "x = " << x << ", y = " << y << ", z = " << z << endl;
  return 0;
}
```

```
$ ./a.out
insert a character, a real, and an integer
c 1.1 2
```

{../L3-4_EXAMPLE_stream/example_cin.cc}

Input stream – examples (1)

```
using namespace std;
#include <iostream>

int main()
{
  char x;
  float y;
  int z;

  cout << "insert a character, a real, and an integer" << endl;
  cin >> x;
  cin >> y;
  cin >> z;
  cout << "x = " << x << ", y = " << y << ", z = " << z << endl;
  return 0;
}
```

```
$ ./a.out
insert a character, a real, and an integer
c 1.1 2
x = c, y = 1.1, z = 2
```

{../L3-4_EXAMPLE_stream/example_cin.cc}

Input stream – examples (1)

```
using namespace std;
#include <iostream>

int main()
{
  char x;
  float y;
  int z;

  cout << "insert a character, a real, and an integer" << endl;
  cin >> x;
  cin >> y;
  cin >> z;
  cout << "x = " << x << ", y = " << y << ", z = " << z << endl;
  return 0;
}
```

```
$ ./a.out
insert a character, a real, and an integer
c 1.1 2
x = c, y = 1.1, z = 2
```

```
$ ./a.out
insert a character, a real, and an integer
c
1.1
2
x = c, y = 1.1, z = 2
```

{../L3-4_EXAMPLE_stream/example_cin.cc}

Input stream – examples (1)

```
using namespace std;
#include <iostream>

int main()
{
  char x;
  float y;
  int z;

  cout << "insert a character, a real, and an integer" << endl;
  cin >> x;
  cin >> y;
  cin >> z;
  cout << "x = " << x << ", y = " << y << ", z = " << z << endl;
  return 0;
}
```

```
$ ./a.out
insert a character, a real, and an integer
2
23
e
x = 2, y = 23, z = 0
```



Source of problems

{../L3-4_EXAMPLE_stream/example_cin.cc}

Input stream – examples (1)

```
using namespace std;
#include <iostream>

int main()
{
  char x;
  float y;
  int z;

  cout << "insert a character, a real, and an integer" << endl;
  cin >> x;
  cin >> y;
  cin >> z;
  cout << "x = " << x << ", y = " << y << ", z = " << z << endl;
  return 0;
}
```

```
$ ./a.out
insert a character, a real, and an integer
c b 1
x = c, y = 0, z = 32765
```

Error state for the stream

Source of problems

```
$ ./a.out
insert a character, a real, and an integer
c b 1.1
x = 2, y = 0, z = 32764
```

Error state for the stream

Source of problems

{../L3-4_EXAMPLE_stream/example_cin.cc}

Input stream – examples (1)

```
using namespace std;
#include <iostream>

int main()
{
  char x;
  float y;
  int z;

  cout << "insert a character, a real, and an integer" << endl;
  cin >> x;
  cin >> y;
  cin >> z;
  cout << "x = " << x << ", y = " << y << ", z = " << z << endl;
  return 0;
}
```

```
$ ./a.out
insert a character, a real, and an integer
A3.5 3
x = A, y = 3.5, z = 3
```

```
$ ./a.out
insert a character, a real, and an integer
3.5 4 3
x = 3, y = 0.5, z = 4
```

{../L3-4_EXAMPLE_stream/example_cin.cc}

Input stream – examples (2)

```
using namespace std;
#include <iostream>

int main()
{
  char x;
  float y;
  int z;

  cout << "insert a character, an integer, and a real" << endl;
  cin >> x;
  cin >> z;
  cin >> y;
  cout << "x = " << x << ", z = " << z << ", y = " << y << endl;
  return 0;
}
```

```
$ ./a.out
insert a character, an integer, and a real
c 2 1.1
x = c, z = 2, y = 1.1
```

```
$ ./a.out
insert a character, an integer, and a real
c 1.1 2
x = c, z = 1, y = 0.1
```

{../L3-4_EXAMPLE_stream/example_cin2.cc}

Input stream – examples (2)

```
using namespace std;
#include <iostream>

int main()
{
  char x;
  float y;
  int z;

  cout << "insert a character, an integer, and a real" << endl;
  cin >> x;
  cin >> z;
  cin >> y;
  cout << "x = " << x << ", z = " << z << ", y = " << y << endl;
  return 0;
}
```

```
$ ./a.out
insert a character, an integer, and a real
3.5 4 3
x = 3, z = 0, y = 4
```

```
$ ./a.out
insert a character, an integer, and a real
3 0.5
x = 3, z = 0, y = 0.5
```

{../L3-4_EXAMPLE_stream/example_cin2.cc}

Some useful functions of the library *<iostream>*

- cin.eof(): it returns a value different from 0 if the stream cin has reached the end (end of stream)
  - It needs to be used always after, at least, a reading operation
- cin.fail(): it returns a value different from 0 if the stream cin has revealed an error state (e.g., string for int) or an end-of-stream
  - It is not necessarily used after, at least, one reading operation
  - It does not require a separator after the last element read
- cin.clear(): it restores the normal state from the error state
- c.ignore(*n*): it ignores or cleans *n* characters from the stream; if n is not specified, then n=1

Example of Reading from *cin* with a failure (1)

```
int main ()
{
  int x1;
  int x2;

  cout << "Insert two integer numbers" << endl;
  cin >> x1;
  cin >> x2;
  cout << "x1 = " << x1 << ", x2 = " << x2 << endl;

  return 0;
}
```

```
$ ./a.out
insert two integers
1 2
x1 = 1, x2 = 2
```

```
$ ./a.out
insert two integers
c 2
x1 = 0, x2 = 32765
```

Strange behavior: error state for the stream

```
$ ./a.out
insert two integers
1 c
x1 = 1, x2 = 0
```

Strange behavior: error state for the stream

{../L3-4_EXAMPLE_stream/example_cin_nofailure.cc}

Example of Reading from *cin* with a failure (2)

```
1.  int main ()
2.  {
3.    int x1;
4.    int x2;
5.
6.    cout << "Insert two integer numbers" << endl;
7.    cin >> x1;
8.    if (!cin.fail()) {
9.      cin >> x2;
10.     if (!cin.fail()) {
11.       cout << "x1 = " << x1 << ", x2 = " << x2 << endl;
12.     }
13.     else
14.       cout << "Error while reading x2" <<endl;
15.   }
16.   else
17.     cout << "Error while reading x1" <<endl;
18.
19.   return 0;
20. }
```

cin.fail() returns a value different from 0 if the stream cin has revealed an error state

If (not cin.fail()) → If (cin.fail()==0) → if there is no error

else branches are executed if there is an error while reading x1 or x2

```
$ ./a.out
insert two integers
1 2
x1 = 1, x2 = 2
```

```
$ ./a.out
insert two integers
c 2
Error while reading x1
```

```
$ ./a.out
insert two integers
1 c
Error while reading x2
```

{../L3-4_EXAMPLE_stream/example_cin_failure.cc}

Example of Reading from *cin* with a failure (3)

```
1.  int main ()
2.  {
3.    int x1, x2;
4.    cout << "Insert two integer numbers" << endl;
5.    cin >> x1;
6.    if (!cin.fail()) {
7.       cin >> x2;
8.       if (!cin.fail()) {
9.          cout << "x1 = " << x1 << ", x2 = " << x2 << endl;
10.      }
11.      else { ... }
12.   else {
13.     cout << "Error while reading x1" <<endl;
14.     cin >> x2;
15.     cout << "x1 = " << x1 << ", x2 = " << x2 << endl;
16.   }
17.   return 0;
18. }
```

```
$ ./a.out
insert two integers
c 2
Error while reading x1
x1 = 0, x2 = 0
```

Strange behavior (error state)

In case of error while reading x1 or x2, we try to read the next int

See the full example: {../L3-4_EXAMPLE_stream/example_cin_failure2.cc}

Example of Reading from *cin* with a failure (3a)

```
1.  int main ()
2.  {
3.    int x1, x2;
4.    cout << "Insert two integer numbers" << endl;
5.    cin >> x1;
6.    if (!cin.fail()) {
7.       cin >> x2;
8.       if (!cin.fail()) {
9.          cout << "x1 = " << x1 << ", x2 = " << x2 << endl;
10.      }
11.      else { … }
12.   else {
13.     cout << "Error while reading x1" <<endl;
14.     cin.clear();
15.     cin.ignore();
16.     cin >> x2;
17.     cout << "x1 = " << x1 << ", x2 = " << x2 << endl;
18.   }
19.   return 0;
20. }
```

```
$ ./a.out
insert two integers
c 2
Error while reading x1
x1 = 0, x2 = 2
```

In case of error while reading x1 or x2, to avoid further errors and read the next int, we have to clear and ignore

See the full example: {../L3-4_EXAMPLE_stream/example_cin_failure2.cc}

Reading a boolean from *cin*

Let's consider the instruction cin >> x, where x is of type bool

␣␣␣␣0␣␣1␣␣7␣true␣ …



(where, ⬆ is the cursor and ␣ is a space)

- If the current position pointed by the cursor represents a space:
  1. the cursor moves forward in order to find a character that is not a space
- If the character pointed by the cursor does not represent a space:
  1. the character is picked up
  2. the character is converted in a boolean and assigned to the variable x (of type boolean)
     - If the character is 0 o 1 the conversion works as expected
     - If the character is different than 0 or 1 (e.g., a digit greater than 1 or a char), the conversion fails silently and the reading operation from the cin is interrupted (not the program execution),
       - If character is a digit greater than 1, x is set to 1
       - If character is a character, x is set to 0
  3. the pointer moves to the next one

Example of Reading a boolean from *cin*

```
bool b1;
bool b2;

cout << "insert in sequence two boolean (0 or 1)" << endl;

cin >> b1;
cin >> b2;
cout << "b1 = " << b1 << ", b2 = " << b2 << endl;
```

b1	b2	cout	cin status
0	1	b1=0, b2=1	
1	0	b1=1, b2=0	
0	5	b1=0, b2=1	err
4	0	b1=1, b2=0	err
t	r	b1=0, b2=0	err
t	0	b1=0, b2=0	err
0	t	b1=0, b2=0	err

You cannot enter the second input, since a silent failure occurs → You have to use fail(), clear() and ignore() to manage this situation

Example of Boolean

{../L3-4_EXAMPLE_stream/example_cinB.cc}

Examples

Example of use of input operations

{../L3-4_EXAMPLE_stream/example_cin.cc}

Example with multiple input

{../L3-4_EXAMPLE_stream/example_cin2.cc}

Example of with integer and then real number

{../L3-4_EXAMPLE_stream/example_cin3.cc}

{../L3-4_EXAMPLE_stream/example_cin4.cc}

Example of with integer and then real number

{../L3-4_EXAMPLE_stream/example_cin_nofailure.cc}

{../L3-4_EXAMPLE_stream/example_cin_failure.cc}

{../L3-4_EXAMPLE_stream/example_ cin_failure2.cc}

Example of use of input operations

{../L3-4_EXAMPLE_stream/example_cinB.cc}